⚙ MAILDORA ⚙

System Architecture

NestJS · BullMQ · Redis · PostgreSQL · Docker · Nginx

Complete Technical Design Document | v2.0 | March 2026

Context

1. High-level System Overview
2. Monorepo Structure
3. Container Architecture (Docker)
4. Request Flow Diagram
5. Email Queue & Worker Architecture
6. Database Schema Design
7. Authentication & Security Flow
8. API Endpoint Map
9. Scaling Strategy

1.SYSTEM OVERVIEW

Maildora একটি multi-tier, event-driven Email Campaign SaaS Backend। System টি তিনটি independent layer এ বিভক্ত : HTTP Layer, Queue Layer এবং Processing Layer।

SYSTEM OVERVIEW

```
┌──────────────────────────────────────┐
│            INTERNET / CLIENT         │
└──────────────────────────────────────┘
                  │ HTTP Request
                  ▼
┌──────────────────────────────────────┐
│          NGINX  (port 80)            │
│       Reverse Proxy / Load Balancer  │
└──────────────────────────────────────┘
          │                │
  /api/*  │                │  /*
          ▼                ▼
┌──────────────────┐  ┌──────────────────┐
│  API SERVER      │  │  WEB SERVER      │
│  NestJS :4000    │  │  Next.js :3000   │
│  (HTTP only)     │  │  (Frontend)      │
└──────────────────┘  └──────────────────┘
        │ Add Job
        ▼
┌──────────────────┐
│  REDIS           │
│  BullMQ Queue    │
│  (Job Broker)    │
└──────────────────┘
        │ Pick Job
        ▼
┌──────────────────┐          ┌──────────────────┐
│  WORKER SERVER   │──────────►│  BREVO API       │
│  NestJS Worker   │  SMTP    │  Email Provider  │
│  (Email Processor)│         └──────────────────┘
└──────────────────┘
        │ Read/Write
        ▼
┌──────────────────┐
│  POSTGRESQL      │
│  Neon (Serverless)│
│  + PgBouncer Pool │
└──────────────────┘
```

2.MONOREPO DIRECTORY STRUCTURE

```
maildora_email_campaign/
│
├── apps/
│   ├── api/                    // NestJS HTTP Server
│   │   ├── src/
│   │   │   ├── modules/
│   │   │   │   ├── auth/       // ClerkAuthGuard, RolesGuard
│   │   │   │   ├── campaigns/  // Campaign CRUD + Send
│   │   │   │   ├── contacts/   // Contact CRUD + Bulk Import
│   │   │   │   ├── queue/      // QueueModule, Bull Board
│   │   │   │   ├── sender-emails/       // Organization + Members
│   │   │   │   └── webhooks/   // Clerk Webhook Sync
│   │   │   ├── prisma.service.ts
│   │   │   └── main.ts
│   │   ├── prisma/
│   │   │   └── schema.prisma   // Database Schema
│   │   └── Dockerfile
│   │
│   ├── web/                    // Next.js Frontend
│   │   └── Dockerfile
│   │
│   └── worker/                 // NestJS Worker Service
│       ├── src/
│       │   ├── processors/
│       │   │   ├── priority.processor.ts  // concurrency: 5
│       │   │   ├── bulk.processor.ts      // concurrency: 3
│       │   │   ├── retry.processor.ts     // concurrency: 2
│       │   │   └── email.sender.ts        // Shared Brevo logic
│       │   ├── worker.module.ts
│       │   ├── prisma.service.ts
│       │   └── main.ts
│       └── Dockerfile
│
├── packages/
│   └── queue/                  // Shared Queue Constants + Types
│       └── src/
│           ├── index.ts        // TypeScript types (local dev)
│           └── index.js        // JS (Docker build)
│
├── docker/
│   └── nginx/
│       ├── nginx.conf
│       └── Dockerfile
│
├── docker-compose.yml
├── package.json
├── pnpm-workspace.yaml
└── turbo.json
```

3. DOCKER CONTAINER ARCHITEC

```
▌ DOCKER COMPOSE — Container Network ▌

┌──────────────────────────────────────────────────────────────────────┐
│                  maildora_network  (bridge)                          │
│                                                                      │
│  ┌──────────────┐      ┌──────────────┐      ┌──────────────┐       │
│  │  maildora_   │      │  maildora_   │      │  maildora_   │       │
│  │  nginx       │      │  api         │      │  web         │       │
│  │              │      │              │      │              │       │
│  │ port: 80     │──────►│ port: 4000   │      │ port: 3000   │       │
│  │ nginx:alpine │      │ node:20-     │      │ node:20-     │       │
│  │              │──────►│ alpine       │      │ alpine       │       │
│  │ depends on:  │      │              │      │              │       │
│  │ api          │      │ healthcheck  │      │ healthcheck  │       │
│  │ web          │      │ /health      │      │ /            │       │
│  └──────────────┘      └──────────────┘      └──────────────┘       │
│                                  │                                   │
│                                  │                                   │
│  ┌──────────────┐      ┌──────────▼──────────┐                      │
│  │  maildora_   │      │  maildora_          │                      │
│  │  worker      │      │  redis              │                      │
│  │              │      │                     │                      │
│  │ node:20-     │◄─────│ redis:7-            │                      │
│  │ alpine       │      │ alpine              │                      │
│  │              │      │                     │                      │
│  │ cpus: 0.5    │      │ port: 6379          │                      │
│  │ mem: 512MB   │      │ healthcheck         │                      │
│  └──────────────┘      └─────────────────────┘                      │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                          External Services
                                  │
              ┌───────────────────┴───────────────────┐
              │                                       │
          ┌───▼────┐                              ┌───▼────┐
          │ Neon   │                              │ Brevo  │
          │ Postgres│                             │ Email  │
          │ + Pgbncr│                             │ API    │
          └────────┘                              └────────┘
```

Container	Image	Details
maildora_nginx	nginx:alpine	Reverse proxy : port 80 expose
maildora_api	node:20-alpine	NestJS API : port 4000 (internal)
maildora_web	node:20-alpine	Next.js : port 3000 (internal)
maildora_worker	node:20-alpine	Worker : no port, only Redis access
maildora_redis	redis:7-alpine	BullMQ broker : port 6379 (internal)

4. REQUEST FLOW DIAGRAMS

4.1 Normal API Request Flow

```
▌ API REQUEST FLOW — e.g. GET /api/campaigns ▌

Client (Browser / Postman)
  |
  |  HTTP GET http://localhost/api/campaigns
  ▼
┌──────────────────────────────────────────┐
│  NGINX                                   │
│  location /api/ {                        │
│    rewrite ^/api/(.*) /$1 break;         │
│    proxy_pass http://api:4000;           │
│  }                                       │
└──────────────────────┬───────────────────┘
                       │  GET /campaigns
                       ▼
┌──────────────────────────────────────────┐
│  NestJS API  (:4000)                     │
│                                          │
│  1. ClerkAuthGuard → verify JWT token    │
│  2. RolesGuard     → check ADMIN/MEMBER  │
│  3. Controller     → call Service        │
│  4. Service        → Prisma query        │
│  5. Return JSON response                 │
└──────────────────────┬───────────────────┘
                       │  SELECT * FROM campaigns
                       │  WHERE organizationId = ?
                       ▼
┌──────────────────────────────────────────┐
│  POSTGRESQL (Neon)                       │
│  Connection via PgBouncer pooler         │
│  max connections: 112                    │
└──────────────────────────────────────────┘
                       │  Return rows
                       ▼
             JSON Response → Client
```

4.2 Campaign Send Flow

```
┃ CAMPAIGN SEND FLOW — POST /campaigns/:id/send ┃

Client
  │  POST /api/organizations/:orgId/campaigns/:id/send
  ▼
  NGINX → API (:4000)
  │
  │  Step 1: Validate campaign (status = DRAFT?)
  │  Step 2: Load all active contacts of organization
  │  Step 3: Create Recipient records (status: PENDING)
  │  Step 4: Update Campaign status → PROCESSING
  │
  │  Step 5: Build jobs array
  │  ┌──────────────────────────────────────────┐
  │  │ jobs = contacts.map(contact => ({         │
  │  │   name: 'send-campaign-email',            │
  │  │   data: { campaignId, recipientId,        │
  │  │           to, subject, body, ... }        │
  │  │ }))                                       │
  │  └──────────────────────────────────────────┘
  │
  │  Step 6: queue.addBulk(jobs)
  ▼
┌──────────────────────────────────────────────────┐
│  REDIS (BullMQ)                                  │
│                                                  │
│  priority-queue ──────────► [job][job][job]      │
│  bulk-queue     ──────────► [job][job][job]      │
│  retry-queue    ──────────► [job][job]           │
└──────────────────────────────────────────────────┘
                        │
                        │  Worker picks jobs
                        ▼
┌──────────────────────────────────────────────────┐
│  WORKER SERVICE                                  │
│                                                  │
│  PriorityProcessor (concurrency: 5)              │
│    → Update Recipient: QUEUED                    │
│    → Call Brevo API                              │
│    → Success: Update Recipient: SENT             │
│               Update CampaignStats.totalSent++   │
│    → Fail:    Update Recipient: FAILED           │
│               Update CampaignStats.failed++      │
│               Re-queue to retry-queue            │
└──────────────────────────────────────────────────┘
                        │
          ┌─────────────┴──────────────┐
          ▼                            ▼
    ┌──────────┐                ┌────────────────┐
    │ Brevo API │                │ PostgreSQL     │
    │ (Email)   │                │ (Status log)   │
    └──────────┘                └────────────────┘
```

5. EMAIL QUEUE & WORKER ARCHITECTURE

Multi-Queue System

```
▌ MULTI-QUEUE SYSTEM DESIGN ▌

                    QUEUE ROUTER
                (campaigns.service.ts)
                          |
          ┌───────────────┼───────────────┐
          |               |               |
          ▼               ▼               ▼

┌────────────────┐  ┌────────────────┐  ┌────────────────┐
| priority-      |  | bulk-          |  | retry-         |
| queue          |  | queue          |  | queue          |
|                |  |                |  |                |
| Rate: 150/min|  | Rate: 150/min|  | No rate limit|
| Retry: 3x      |  | Retry: 3x      |  | Retry: 5x      |
| Backoff: 5s    |  | Backoff: 5s    |  | Backoff: 30s   |
└────────────────┘  └────────────────┘  └────────────────┘
        |                |                |
        ▼                ▼                ▼

┌────────────────┐  ┌────────────────┐  ┌────────────────┐
| Priority       |  | Bulk           |  | Retry          |
| Processor      |  | Processor      |  | Processor      |
|                |  |                |  |                |
| concurrency:5|  | concurrency:3|  | concurrency:2|
└────────────────┘  └────────────────┘  └────────────────┘
        └────────────────┼────────────────┘
                         |
                         ▼
                  email.sender.ts
                (Shared Brevo Logic)
                         |
                         ▼
                     Brevo API
                max: 300 emails/min
          (150 priority + 150 bulk = 300 total)
```

Job Lifecycle

```
▌ JOB STATUS TRANSITIONS ▌

                    addBulk()
                       │
                       ▼
              ┌────────────────┐
              │    WAITING     │  ← Queue তে add হওয়ার পর
              └────────────────┘
                       │  Worker picks up
                       ▼
              ┌────────────────┐
              │    ACTIVE      │  ← Worker process করছে
              └────────────────┘
                       │
          ┌────────────┴────────────┐
          │                         │
          ▼ Success                 ▼ Fail
   ┌────────────────┐       ┌──────────────────┐
   │   COMPLETED    │       │      FAILED      │
   │                │       │                  │
   │ removeOn:      │       │  attempts < max? │
   │ Complete:100   │       │        │         │
   └────────────────┘       │   Yes  │  No     │
                            │        │         │
                            ▼        ▼         │
                         ┌─────────┐ ┌────────┐ │
                         │DELAYED  │ │ FAILED │ │
                         │(retry)  │ │(stored)│ │
                         └─────────┘ └────────┘ │
                              │       removeOn:  │
                              │       Fail: 50   │
                         exponential            │
                         backoff wait           │
                              │                 │
                              └─────────────────┘
                                    retry
```

Status	Log	Details
waiting	Successfully added to Queue	Normal state
active	Processing by Worker	Locked
completed	Successfully sent	Contains max 100 campaign
failed	Max retry , then fail	Contains max 50 campaign
delayed	Backoff wait — before retry	Exponential: 5s→25s→125s

6. DATABASE SCHEMA DESIGN

```
▌ PRISMA SCHEMA — Entity Relationship ▌

┌──────────────┐          ┌──────────────────────┐
│    User      │          │    Organization      │
│──────────────│          │──────────────────────│
│ id (PK)      │          │ id (PK)              │
│ clerkId      │          │ name                 │
│ email        │          │ createdAt            │
│ firstName    │          └──────────────────────┘
│ lastName     │                     │ 1
└──────────────┘                     │
        │                            │ many
        │ many                       ▼
        └──────────────────┌──────────────────────┐
                           │  OrganizationMember  │
                           │──────────────────────│
                           │ id (PK)              │
                           │ userId (FK)          │
                           │ organizationId (FK)  │
                           │ role: ADMIN/MEMBER   │
                           │ @@unique([userId,    │
                           │          orgId])     │
                           └──────────────────────┘

┌──────────────────┐      ┌──────────────────────┐
│    Campaign      │      │      Contact         │
│──────────────────│      │──────────────────────│
│ id (PK)          │      │ id (PK)              │
│ organizationId   │      │ organizationId       │
│ name             │      │ email                │
│ subject          │      │ firstName            │
│ body             │      │ lastName             │
│ status: DRAFT/   │      │ isActive             │
│  PROCESSING/SENT │      │ @@unique([email,     │
│ senderEmailId    │      │           orgId])    │
└──────────────────┘      └──────────────────────┘
         │ 1
         │ many
         ▼
┌──────────────────┐      ┌──────────────────────┐
│    Recipient     │      │    CampaignStats     │
│──────────────────│      │──────────────────────│
│ id (PK)          │      │ id (PK)              │
│ campaignId (FK)  │      │ campaignId (FK, 1:1) │
│ contactId (FK)   │      │ totalSent            │
│ email            │      │ failed               │
│ status: PENDING/ │      │ pending              │
│  QUEUED/SENT/    │      │ opened               │
│  FAILED          │      │ clicked              │
│ sentAt           │      │                      │
└──────────────────┘      └──────────────────────┘

┌──────────────────┐
│   SenderEmail    │
│──────────────────│
│ id (PK)          │
│ organizationId   │
│ email            │
│ isVerified       │
└──────────────────┘
```

7. AUTHENTICATION & SECURITY FLOW

```
▌ AUTH FLOW — Clerk Integration ▌

// ----- Sign Up Flow -------

User signs up on Clerk
  |
  |  Clerk fires webhook: user.created
  ▼
POST /webhooks/clerk
  |
  |  1. Svix: verify webhook signature
  |     (prevents fake webhooks)
  |
  |  2. Prisma transaction:
  |     → Create User
  |     → Create Organization (name from email)
  |     → Create OrganizationMember (role: ADMIN)
  ▼
  Done — User is ready to use API

// --- Protected API Request Flow ------------------------

Client sends: Authorization: Bearer <clerk_jwt>
  |
  ▼
ClerkAuthGuard
  |  → verifyToken(jwt) via Clerk SDK
  |  → Load User from DB by clerkId
  |  → Attach user to request
  |
  ▼
RolesGuard
  |  → Check @Roles() decorator on route
  |  → Load OrganizationMember record
  |  → Verify user has required role
  |
  ▼
Controller → Service
  |  → All queries include organizationId
  |  → Data isolation guaranteed
  ▼
  Response
```

Mechanism	কোথায়	কাজ
ClerkAuthGuard	সব protected route এ	JWT verify + DB user load
RolesGuard	@Roles() decorator দেওয়া routes এ	ADMIN/MEMBER check
Svix	Webhook endpoint এ	Clerk signature verify
organizationId	সব Service method এ	Data isolation enforce

8. COMPLETE API ENDPOINT MAP

Public / Webhook

Endpoint	Details
GET /health	API health check — Docker healthcheck use করে
POST /webhooks/clerk	Clerk user.created webhook — Svix verified

Organization

Endpoint	Details
POST /organizations	Organization create
GET /organizations/:id	Organization details
POST /organizations/:id/members	Member invite (ADMIN only)
GET /organizations/:id/members	Member list

Contacts

Endpoint	Details
GET /organizations/:id/contacts	Contacts list — pagination + search
POST /organizations/:id/contacts	Single contact add
POST /organizations/:id/contacts/import	Bulk import (100/batch)
PATCH /organizations/:id/contacts/:cid	Contact update
DELETE /organizations/:id/contacts/:cid	Contact delete

Campaigns

Endpoint	Details
GET /organizations/:id/campaigns	Campaign list + pagination
POST /organizations/:id/campaigns	Campaign তৈরি (DRAFT status)
GET /organizations/:id/campaigns/:cid	Campaign details + stats
PATCH /organizations/:id/campaigns/:cid	Campaign update (DRAFT only)
POST /organizations/:id/campaigns/:cid/send	Campaign queue এ add করে send
POST /organizations/:id/campaigns/:cid/duplicate	Campaign duplicate
PATCH /organizations/:id/campaigns/:cid/cancel	Campaign cancel

DELETE /organizations/:id/campaigns/:cid	Campaign delete (DRAFT only)

Sender Emails

Endpoint	Details
GET /organizations/:id/sender-emails	Verified sender list
POST /organizations/:id/sender-emails	Sender add → Brevo verification
POST /organizations/:id/sender-emails/:sid/sync	Verification status sync
DELETE /organizations/:id/sender-emails/:sid	Sender delete

Monitor

Endpoint	Details
GET /admin/queues	Bull Board — real-time queue dashboard

9. SCALING STRATEGY

```
▌ HORIZONTAL SCALING — Worker Scale-out ▌

// Current (1 worker):
docker compose up -d

// Scale to 3 workers:
docker compose up --scale worker=3 -d

// Result with 3 workers:

        REDIS Queue
             |
     ┌───────┼───────┐
     ▼       ▼       ▼
┌────────┐ ┌────────┐ ┌────────┐
│Worker 1│ │Worker 2│ │Worker 3│
│        │ │        │ │        │
│P:5     │ │P:5     │ │P:5     │
│B:3     │ │B:3     │ │B:3     │
│R:2     │ │R:2     │ │R:2     │
└────────┘ └────────┘ └────────┘

// Total concurrent: 3 × 10 = 30 jobs at once
// But Brevo limit = 300/min
// So effectively: 300 emails/min regardless of workers
// Workers help when Brevo limit is higher (paid plan)
```

Scaling Matrix

Brevo Plan	Workers	Throughput
Free (300/min)	1-3 workers	~300 emails/min
Starter (2,000/min)	3-5 workers	~2,000 emails/min
Business (10,000/min)	10-15 workers	~10,000 emails/min

Nginx → Load Balancer Upgrade

```
# Current (reverse proxy)
set $api_upstream http://api:4000;

# Upgraded (load balancer)
upstream api_cluster {
    least_conn;
    server api1:4000;
    server api2:4000;
    server api3:4000;
}
```